SMART STOCK ANALYZER

Project Flaw Analysis & Improvement Report

Review of the Phase-II Progress Report and both Research Paper versions

Project	Smart Stock Analyzer
Team	Code Geeks
Project focus	Portfolio tracking, simulated stock trading, risk scoring, DBMS integrity and OS concurrency
Review basis	Three supplied PDFs only; no source-code audit was performed

Purpose: Identify weaknesses that could affect correctness, security, scalability, credibility of results, and project evaluation. Findings are labelled as documented gaps, internal inconsistencies, or items requiring code-level verification.

1. Executive Summary

The project has a useful educational scope and its reports describe sound goals: atomic trade processing, relational integrity, portfolio calculations, and visible OS concurrency concepts. The main weakness is the gap between the target architecture described in the papers and the implementation status recorded in the Phase-II report. The progress report says the project is around 60–65% complete and lists the Java Swing client, broader REST API, MySQL migration, security hardening, automated testing and benchmarks as pending.

The most important risks are: (1) architecture and implementation descriptions do not consistently match; (2) the risk score and trading restriction rules are not validated against real outcomes; (3) the research-paper performance claims need reproducible scripts and fuller statistical reporting; (4) security controls are explicitly still pending in the progress report; and (5) transaction, accounting and concurrency behaviour need broader edge-case tests.

Scope note: This is a document-based review, not proof that every suspected defect exists in the code. Where the PDFs do not provide enough evidence, the report recommends a verification test rather than declaring a confirmed bug.

2. Highest-Priority Findings

Critical / P0 — Core architecture is described as complete before it is complete

Evidence in supplied reports: The Phase-II report says the current prototype uses SQLite on the web and that only /api/prices exists; the REST API expansion, Java Swing client and SQLite-to-MySQL migration are pending. Yet the abstract and architecture sections describe a Java Swing + Flask multi-tier solution as if it were the current system.

Recommended fix: This can mislead evaluators about what is implemented versus planned. Label every diagram and feature as Implemented, Partial, or Planned; update the abstract and architecture to reflect the actual build.

High / P1 — Security hardening is incomplete

Evidence in supplied reports: The progress report explicitly lists CSRF protection and moving the secret key to environment variables as pending. The report also describes login/session handling, but the supplied PDFs do not establish that session-cookie settings, rate limiting, password-reset handling, or authorization checks have been comprehensively tested.

Recommended fix: Before any deployment beyond a local demo, implement CSRF protection where cookie-authenticated state-changing requests are used, load secrets from environment configuration, set secure session-cookie flags in production, and test access-control boundaries. Do not claim production readiness.

High / P1 — Risk score is a heuristic, not a validated risk model

Evidence in supplied reports: The v2 paper defines $s = 0.5 \cdot \min(\sigma/0.8,1) + 0.5 \cdot \min(D/0.6,1)$, using volatility and maximum drawdown with fixed caps and weights. It acknowledges synthetic price paths and says the score has not been validated against realised outcomes.

Recommended fix: A score threshold can look authoritative without being predictive. Explain units, lookback window, annualisation, missing-data behaviour and threshold rationale. Validate on held-out historical data before making any performance or investment-quality claim.

High / P1 — Risk score controls account access/trading rather than only communicating risk

Evidence in supplied reports: The four bands include a restriction mode that disables trading above 0.85. The papers do not demonstrate that this threshold is appropriate, fair across assets, or robust to data errors.

Recommended fix: For an educational app, label this as a demonstration rule. Separate risk warnings from account authorization and provide a clear explanation, override/governance policy, and tests at exact boundary values.

High / P1 — Concurrency claims need a reproducible implementation package

Evidence in supplied reports: The v2 paper reports 16 concurrent threads, 6,580 inconsistent shares on average for an intentionally unsynchronised variant, and zero inconsistency in 25 atomic runs. It also states the prototype uses SQLite and one connection per thread.

Recommended fix: The result supports the lesson but is specific to a synthetic workload and a single machine. Publish the exact benchmark code, seed, schema, transaction boundaries, SQLite settings, and command to reproduce it. Test concurrent buys and sells, same-user/same-stock contention, failures mid-transaction, and retries.

High / P1 — SQLite write serialization limits scalability

Evidence in supplied reports: The Phase-II report itself notes SQLite permits only one writer at a time and that the background price updater can delay trades. MySQL migration and row-level locking are pending.

Recommended fix: Avoid presenting the current build as a high-throughput multi-user trading architecture. Measure wait time and throughput under mixed price-update/trade load; after migration, repeat benchmarks using appropriate isolation and row locks.

High / P1 — Accounting edge cases are under-specified

Evidence in supplied reports: The progress report specifically calls out partial-sale fees and break-even handling as a challenge. The papers provide average-cost and break-even formulas, but the supplied material does not fully define realised P/L, fee allocation across partial sales, tax/currency treatment, or rounding rules.

Recommended fix: Write a single accounting specification and add worked examples for multiple buys, partial sells, full sells, fees on both sides, zero holdings, and rounding. Compare implementation outputs against hand-calculated expected values.

Medium / P2 — Performance claims lack uncertainty and broader workload coverage

Evidence in supplied reports: The v2 paper reports an 18–28× query-latency improvement from an index at 10k–500k rows, with 200 timed queries, and five repetitions for concurrency configurations. It recommends confidence intervals as future work.

Recommended fix: Report raw measurements, mean and spread for every size, machine/software configuration, warm-up/cache policy, query plan, and confidence intervals. Benchmark more query patterns and mixed read/write workloads; avoid generalising single-node results to production.

Medium / P2 — The indexing evaluation may overstate general benefit

Evidence in supplied reports: The reported index is Transactions(user_id), and the timed query groups transactions by stock and type for one user. This is a reasonable targeted index, but no comparison of composite indexes or write overhead is shown.

Recommended fix: Use EXPLAIN QUERY PLAN and compare (user_id), (user_id, stock_id), and any date-oriented index against actual query patterns. Include insert/update overhead and storage cost before choosing a final index.

Medium / P2 — The decision-band experiment demonstrates reachability, not quality

Evidence in supplied reports: The v2 paper generates 200 synthetic one-year price paths with a deliberately wide volatility range and reports that all four bands are populated. This only proves the thresholds can be exercised.

Recommended fix: Do not describe this as evidence that the system identifies real-world risk correctly. Evaluate calibration, false positives/negatives, stability across market regimes, and sensitivity to the chosen weights and caps.

Medium / P2 — Test coverage is reported as a status list, not a complete test artefact

Evidence in supplied reports: The Phase-II report lists several passing manual-style tests, while the pending-task table still lists the automated test suite and load/concurrency benchmarks. The papers describe tests and methodology, but the PDFs do not provide the complete runnable suite.

Recommended fix: Include test names, setup, expected/actual results, logs, and automated commands. Add boundary tests for scores 0.40, 0.70 and 0.85, plus rollback, duplicate requests, malformed inputs, session expiry and concurrent trades.

Medium / P2 — The database design description is inconsistent across versions

Evidence in supplied reports: The Phase-II report lists a price_history table among six tables and mentions an index; the research-paper schema presents five principal tables (Users, Stocks, Portfolio, Transactions, Logs) and does not consistently describe price_history as a core table. The papers also differ in how Portfolio keys are described.

Recommended fix: Choose one authoritative schema and update the report, ER diagram, table definitions, code and migrations together. Clarify whether price history is persisted or generated in memory, and define the portfolio key consistently.

Medium / P2 — Research novelty and related-work claims are too broad

Evidence in supplied reports: The v2 paper claims the surveyed literature gives little attention to transactional data management and says none of the surveyed studies primarily addresses that layer. The review table is selective and the documents do not show a systematic search protocol for this specific claim.

Recommended fix: Narrow the claim to 'within the reviewed works' unless a reproducible search supports a broader conclusion. Document databases searched, query terms, inclusion/exclusion criteria, and the date range.

Medium / P2 — Operational resilience and data lifecycle are not fully addressed

Evidence in supplied reports: The documents discuss audit logs and an append-only transaction ledger, but do not establish backup/restore procedures, schema migration strategy, retention policy, monitoring, or recovery after process/database failure.

Recommended fix: Add backup-and-restore validation, migration scripts, structured error logging, health checks, and a recovery test that kills the process during a trade and confirms consistent state after restart.

Low / P3 — Terminology and presentation can confuse evaluators

Evidence in supplied reports: The progress report uses 'Smart Stock Analyzer' while the abstract calls it 'SSA Pro'; the documents mix planned architecture with present-tense implementation statements. Some OS concepts such as process-level parallelism are listed as pending while broader text describes processes/worker pools as implemented.

Recommended fix: Use one project name. Distinguish threads from processes precisely and map each OS concept to code evidence, file/function name, and a test or measurement.

3. Cross-Document Inconsistencies

Topic	What the PDFs say	Why it matters / action
Architecture	Phase-II: browser prototype; only /api/prices; Swing client and full REST API pending. Papers: web/desktop frontend and layered API are described as system architecture.	Mark the target architecture separately from the current implementation.
Database	Phase-II mentions users, stocks, price_history, portfolio, transactions and logs; papers list five principal tables and do not consistently include price_history.	Publish one schema/ERD and use it consistently in all documents.
OS parallelism	Phase-II lists process-level multiprocessing for risk computation as pending; papers discuss worker processes/pools in broad terms.	Only claim process-level parallelism if implemented and measured; otherwise label it future work.
Project naming	Project is titled Smart Stock Analyzer, while the Phase-II abstract calls it “SSA Pro.”	Standardize the name throughout reports, UI, repository and diagrams.
Testing	Phase-II reports a short list of passed tests but says automated tests and load/concurrency benchmarks are pending; v2 contains detailed experimental claims.	Separate functional tests from research experiments and attach reproducible test artefacts.
Current vs target DB	Phase-II states SQLite is current and MySQL migration is pending; papers discuss row-level locking as a server-DBMS target and SQLite BEGIN IMMEDIATE in the prototype.	Label each concurrency mechanism as implemented in SQLite or proposed for MySQL.

4. What the Documents Do Well

A fair review should preserve the project's strengths: the reports describe a clear educational use case; relational entities and integrity constraints are identified; the trade path is intended to be atomic; the v2 paper makes its synthetic-data limitation explicit; and the concurrency/index experiments are more concrete than unsupported claims alone. The aim is not to discard these contributions, but to make the implementation status, measurements, and limits auditable.

5. Recommended Fix Order

- **1. Reconcile the documentation:** Update abstract, architecture, schema, task status and diagrams so every feature is labelled Implemented / Partial / Planned.
- **2. Finish security basics:** Protect state-changing endpoints, externalise secrets, review session settings, enforce server-side authorization and test common web threats.
- **3. Lock down trade invariants:** Define stock-quantity conservation, non-negative holdings, transaction ledger rules, duplicate-request behaviour and rollback guarantees.
- **4. Validate portfolio maths:** Create a written fee/average-cost/realised-P&L; specification and hand-check all buy/sell combinations.
- **5. Make experiments reproducible:** Commit scripts, fixed seeds, environment details, raw result files and exact run commands; report variability and query plans.
- **6. Finish integration:** Complete REST endpoints and Swing client only after agreeing on the API contract; then perform end-to-end integration tests.
- **7. Benchmark the final DBMS:** After MySQL migration, rerun concurrency and latency tests with mixed workloads and document isolation/locking choices.
- **8. Validate the risk heuristic:** Keep it explicitly educational until tested on appropriate historical data with sensitivity and out-of-sample analysis.
- **9. Add recovery tests:** Verify backup/restore, migration, crash recovery, and consistent state after a process is interrupted during a trade.

6. Questions an Evaluator May Ask

- Which features are running in the current build, and which are only in the proposed architecture?
- Why is SQLite still used if the target is concurrent multi-user trading, and what exactly changes after MySQL migration?
- How is the risk score validated? Why are the caps 0.8 volatility and 0.6 drawdown, and why are the thresholds 0.40, 0.70 and 0.85?
- How are fees allocated when a user sells only part of a position? How is realised profit/loss computed?
- Can the team reproduce the reported 6,580 inconsistent shares and 18–28× index improvement from a clean checkout?
- What tests prove the app remains correct when the background price updater overlaps with simultaneous buy/sell requests?
- What prevents CSRF, session misuse, unauthorized portfolio access and accidental exposure of secrets?
- How does the schema in the report match the schema.sql file and the live database?

7. Overall Assessment

Overall: Promising educational prototype, but the reports do not yet justify describing it as a complete, scalable, production-grade trading system. The strongest next step is to align claims with the actual build, complete security and automated testing, and make the research experiments reproducible. The risk engine should remain clearly labelled as a heuristic until it has been evaluated beyond synthetic data.

Source documents reviewed

1. pbl_phase_2 (2).pdf — Phase-II Project Progress report (6 pages).
2. Smart_Stock_Analyzer_Research_Paper (1).pdf — Research Paper (8 pages).
3. Smart_Stock_Analyzer_Research_Paper_v2.pdf — revised Research Paper (10 pages).

Evidence boundary: Findings are based on the contents of the supplied PDFs. Source-code inspection, live execution, security scanning, and independent replication of benchmark numbers were not performed.